

The `all()` utility returns `True` if all numbers in `spam` or `eggs` are greater than 42.

But if you pass an empty sequence to `all()`, it always returns `True`. Enter the following into the interactive shell:

```
>>> all([])
True
```

It's best to think of `all([])` as evaluating the claim “none of the items in this list are falsey” instead of “all the items in this list are truthy.” Otherwise, you might get some odd results. For instance, enter the following into the interactive shell:

```
>>> spam = []
>>> all([i > 42 for i in spam])
True
>>> all([i < 42 for i in spam])
True
>>> all([i == 42 for i in spam])
True
```

This code seems to be showing that not only are all the values in `spam` (an empty list) greater than 42, but they're also less than 42 and exactly equal to 42! This seems logically impossible. But remember that each of these three list comprehensions evaluates to the empty list, which is why none of the items in them are falsey and the `all()` function returns `True`.

Boolean Values Are Integer Values

Just as Python considers the float value `42.0` to be equal to the integer value `42`, it considers the Boolean values `True` and `False` to be equivalent to `1` and `0`, respectively. In Python, the `bool` data type is a subclass of the `int` data type. (We'll cover classes and subclasses in Chapter 16.) You can use `int()` to convert Boolean values to integers:

```
>>> int(False)
0
>>> int(True)
```
